DHAR

Bare Metal. Absolute Control.

VOLUME III — ECOSYSTEM, TRUST & ADOPTION

TEMPERING DHAR

"Trust that survives contact with an adversary, not just a demo."

DOCUMENT	Tempering Dhar — Security, Assurance & Defence-Grade Engineering
VERSION	1.1
DATE	October 26, 2026
STATUS	Ratified
PART OF	The Canon of Dhar, Volume III
DEPENDS ON	Spec1 §4.7, §11, §14–15, Stage-delta §2–4, Stage-epsilon, Amendments A-1 and A-2
AUDIENCE	Security reviewers, government & defence evaluators

Contents

1. Mandate 3

2. Threat Model 3

2.1 Assets & adversaries 3

2.2 Attack surface 4

3. Memory & Type Safety: What's Eliminated By Construction 4

3.1 What remains the programmer's, or the tooling's, responsibility 5

3.2 The same on every target 5

3.3 Models and files from outside 6

4. Concurrency Safety 6

5. Supply-Chain Security & Provenance 6

5.1 What Kun already guarantees 6

5.2 Reproducible builds as continuous audit 6

5.3 The Trusting Trust problem 7

5.4 Outside implementations are oracles, not dependencies 7

6. Sensitive Data Handling 7

6.1 Constant-time operations 7

6.2 Zero-on-drop 8

6.3 Post-quantum cryptography and algorithm agility 8

7. Certification Readiness 9

8. Vulnerability Disclosure & Incident Response 10

9. Acceptance Criteria 10

10. Relationship to Other Volumes 11

11. Claims and Gates 11

11.1 The rule 11

11.2 The allowed forms 11

11.3 What is never said 12

11.4 When a gate stops passing 12

11.5 Changing the rule 12

1. Mandate

State, precisely and without inflation, what Dhar's design guarantees for security and safety, what it does not, and what would be required to demonstrate either claim to an outside evaluator. Spec0 §5.2 names defence and critical national infrastructure among the domains Dhar serves; this document is where that claim is made accountable rather than aspirational. Nothing here asserts a certification Dhar holds: a certification belongs to a finished, versioned product, and a claim of any kind is allowed only with the evidence of §7 and in the form of §11.

Dhar is a language for trustworthy computation: it goes first to cybersecurity, then to artificial intelligence, then to quantum computing, with every other domain kept as modules (Spec0 §5.1, §5.2; Amendment A-1). This document is where the first of those is made accountable. It states the guarantees that make a program written in Dhar hard to attack, the tests that say whether they hold, and the rule that governs every claim the project makes about them (§11).

2. Threat Model

2.1 Assets & adversaries

ASSET	REPRESENTATIVE ADVERSARY	WHAT THEY WANT
The compiler binary itself	Supply-chain attacker; nation-state actor	Insert a backdoor that survives into every program the compiler builds
Kun's registry & packages	Typosquatter; compromised maintainer account	Distribute malicious code as a trusted dependency
A compiled Dhar program's memory	Remote attacker with network access	Corrupt memory to achieve code execution or data exfiltration
Sensitive runtime data (keys, credentials)	Local attacker; forensic recovery after disposal	Recover secrets from memory, swap, or crash dumps
Cryptographic operations	Adversary with timing/cache measurement access	Recover key material via side channel
Data that is encrypted now	An adversary who records traffic now	Decrypt it later, once a computer able to break the public-key algorithms in use exists (“harvest now, decrypt later”)
Model weight files and datasets	A supplier of a malformed or poisoned file	Corrupt memory in the loader, or change what a model predicts
The hosted compile service (Stage-epsilon §3.1)	Anyone able to send it a request	Escape the sandbox, read another visitor's source, or exhaust the service
One program on many targets	An attacker who looks for the weakest target	Find the processor or operating system on which a guarantee of the language does not hold

2.2 Attack surface

Seven layers, each with a different owner in this Canon:

1. **Compiler-introduced.** The classic case is Ken Thompson's Turing Award lecture, “Reflections on Trusting Trust” (*Communications of the ACM*, 1984): a compiler can be made to insert a backdoor into everything it compiles, including future versions of itself, leaving no trace in the source it was built from. A from-scratch, self-hosting compiler (Stage-alpha through Stage-beta) is exactly the kind of software this attack targets. Addressed in §5.3.
2. **Supply-chain.** A compromised dependency, registry, or build environment. Owned by Kun (Stage-delta §3.7) and Stage-epsilon's reproducible builds and signing; recapped in §5.
3. **Runtime / memory-safety.** Vulnerabilities in compiled Dhar programs themselves — the class of bug Spec0 §2.2's Java-class-robustness commitment targets directly. Addressed in §3.
4. **Side-channel.** Timing, cache, and power-analysis leakage from cryptographic or otherwise sensitive code, which memory safety alone does not address. Addressed in §6.
5. **Cryptographic ageing.** An algorithm that a future computer breaks, and data that was protected by it. Addressed in §6.3.
6. **Hosted service.** A service that compiles code sent by strangers. Owned by Stage-epsilon §3.1; reported under §8.
7. **Target-dependent.** A guarantee that holds on one target and not on another. Addressed in §3.2.

3. Memory & Type Safety: What's Eliminated By Construction

In November 2022 the NSA recommended memory-safe languages; in December 2023, CISA and the NSA jointly published *The Case for Memory Safe Roadmaps*, calling on software manufacturers serving critical infrastructure to publish a memory-safety roadmap; in June 2025 the two agencies followed with practical adoption guidance. Google and Microsoft have both reported that most vulnerabilities in their own products trace back to memory-safety bugs, and Apple's own review of its operating systems found the same pattern. This is not a theoretical concern this Canon is retrofitting a justification for; it is the specific, named problem Dhar's memory model (Spec1 §14) was designed against from the start.

The frequently cited gap in that guidance is real: established memory-safe languages (Go, Java, C#, Python) buy their safety with a garbage collector and a runtime, which is exactly what Spec0 §2.2 rules out for Dhar. Rust and Ada/SPARK are the languages usually named as filling that gap; Dhar's thesis (Spec0 §3) is to occupy the same ground, independently.

Memory safety is the floor of the first track. Each row below states what the design requires; gate S1 (Amendment A-1 §9) is the test that it holds, and §11 gives the form in which it may be asserted.

VULNERABILITY CLASS	CWE	ELIMINATED BY
Buffer overflow (classic, stack, heap)	CWE-120 / 121 / 122	Mandatory bounds checking on every array and raw[N] access (Spec1 §14.4) — an out-of-range index is a diagnosable failure, never silent adjacent-memory access

VULNERABILITY CLASS	CWE	ELIMINATED BY
Out-of-bounds read or write through a slice, view or tensor	CWE-125 / 787	Every index through a view or tensor is bounds-checked, and a view cannot outlive the arena it borrows from (Spec1 §14.4, §14.6)
NULL pointer dereference	CWE-476	own T's absence is a checked None variant (Spec1 §14.3), not an unchecked null; the check is enforced at match time, not left to the programmer to remember
Use-after-free	CWE-416	Compiler-inserted deallocation under single ownership (Spec1 §14.1) — there is no manual free for a use-after-free to follow
Double free	CWE-415	Same mechanism: deallocation is inserted exactly once, by the compiler, tied to the single owner going out of scope
Integer overflow or wraparound	CWE-190	The plain integer operators fail in a safe, diagnosable way when the result does not fit; wrapping, saturating and checked arithmetic are separate, named operations (Spec1 §4.7)
Unchecked return value / ignored error	CWE-252	state must be explicitly trapped (Spec1 §16.1–16.2); an ignored Error case is a compile-time gap, not a silent runtime one
Missing case / incomplete handling	(no single CWE; a root cause behind many)	alloy + match must be exhaustive (Spec1 §11) — a forgotten variant is a compile error

This table is a claim about the language's design, not a claim about any particular program written in it, and not a claim that these CWE categories are the only ones that matter — injection, access-control, and authentication weaknesses (also CWE Top-25 regulars) are application-logic concerns no language design eliminates by itself.

3.1 What remains the programmer's, or the tooling's, responsibility

Every capability that can violate the guarantees above is a named, individually greppable keyword rather than one blanket escape hatch: `peek` / `poke` (raw memory), `asm` (inline assembly), and `foreign` (calling code outside Dhar's guarantees entirely). A secret value (Spec1 §14.5) has a fifth: the explicit reveal that turns it into a public value, which the compiler can list for every build. A security review of a Dhar codebase can locate every place the language's guarantees are suspended with these five search terms, which is a sharper audit surface than a single `unsafe` block covering several unrelated kinds of risk at once (Spec1 §14, §18–19). Logic errors, algorithmic complexity, and injection/access-control bugs are unaffected by any of this — memory safety is necessary for a defence-grade toolchain, not sufficient.

3.2 The same on every target

The guarantees of this section are properties of the language, not of a processor. A target changes the encoding of a program, never its meaning (Amendment A-2 §8). On every committed target (Amendment A-2 §3.1) an out-of-range index stops the program with the same defined message and exit status, an integer operation whose result does not fit fails in the same way, and the rules of §6 hold. A target on which one of them does not hold is not described as supported.

The conformance battery (Stage-alpha §3.3) contains the bounds-check programs and is run for each target at gate G1. The gate that backs a statement about a target is the one named in Amendment A-2 §6. An emulator is evidence of correct output, not of the target's behaviour on real hardware, and is not described as more.

3.3 Models and files from outside

A model's weight file, a dataset, a network message, a source file and a circuit description are all input from outside, and input from outside is hostile until it has been checked (Amendment A-1 §5). A loader written in Dhar reads such a file under the same rules as any other code: every index is checked, and the sizes the file declares are compared with the space the program has set aside, in an arena of fixed size (Spec1 §14.6), before anything is read into it. A file that declares more is refused with an error; it is never truncated and never trusted.

A model's predictions are the program's result, not a security property. This document makes no claim that a model is correct, fair or free of poisoned training data; gate A3 tests only that a network's outputs agree with a reference to a stated tolerance.

4. Concurrency Safety

Spec1 §15 requires that `view` / `grab` exclusivity (Spec1 §14.2) hold across `spark` / `draw` task boundaries, not only within sequential code. Practically: a value cannot be mutated from two concurrently running tasks at once, by the same construction that already prevents two overlapping mutable references in ordinary sequential code. Data races (CWE-362, race condition) are closed as a special case of aliasing, not bolted on as a separate concurrency-checking pass — the same design economy Spec1 §10.5 already applies to logical and symbolic programming.

5. Supply-Chain Security & Provenance

5.1 What Kun already guarantees

Recapping Stage-delta §3.7 rather than restating it: every published package carries a content hash and signature verified on install; resolution is deterministic and lockfiles prevent silent re-resolution; a private, self-hosted registry is first-class, not an afterthought, specifically so a critical-infrastructure adopter's supply chain does not depend on a single externally controlled service.

5.2 Reproducible builds as continuous audit

Stage-epsilon requires two independent builds of the same source, on different machines, to be bit-for-bit identical. This is not only a quality property: it means any third party can independently verify that a released binary actually corresponds to its published source, rather than trusting the publisher's word — the software equivalent of the Common Criteria requirement that evaluators reproduce the evidence themselves rather than accept a vendor's claims.

5.3 The Trusting Trust problem

Stage-beta's self-hosting proof (byte-for-byte identical output across two generations, Stage-beta §4) establishes that Stage-beta reproduces itself correctly; it does not, by itself, prove the compiler is free of a Thompson-style backdoor inserted at the Stage-alpha bootstrap and silently propagated ever since, since a corrupted compiler can reproduce a corrupted copy of itself perfectly. The established academic mitigation is Diverse Double-Compiling (David A. Wheeler, “Countering Trusting Trust through Diverse Double-Compiling”, 2005, and his 2009 doctoral dissertation at George Mason University): compile a compiler's source with a second, independently written compiler, compile the source again with the result, and confirm that this equals the compiler as distributed.

This Canon specifies that check as a gate. Dhar has two independent compilers: Stage-alpha, written in assembly, and Stage-beta, written in Dhar. The check is gate S4 (Amendment A-1 §4, §9; Stage-beta §4.1): Stage-alpha, frozen and archived, compiles Stage-beta's source to give E1; E1 compiles it again to give E2; E2 **must** be byte-for-byte identical to the Stage-beta binary being distributed. The check is repeated for every release of the compiler (Stage-epsilon §3.2). It depends on reproducible builds (§5.2), which is why the two are specified together, and it depends on the archived Stage-alpha being kept exactly as it was frozen (Stage-alpha §5).

What the check shows, and what it does not. Equality shows that the distributed binary corresponds to the published source and carries no change that is not in that source, unless Stage-alpha itself is compromised. It does not show that the source is free of mistakes, or of a flaw written into it on purpose. It rests on a small trusted base: Stage-alpha, the assembler and linker that build it, and the machine it runs on. Dhar aims to keep that base small and auditable (Amendment A-1, P4), and the check **should** be repeated on a second, unrelated machine.

5.4 Outside implementations are oracles, not dependencies

The core language, the compiler and the standard library depend on no outside compiler, runtime or library (Amendment A-1, P1). OpenSSL, libsodium, NumPy, Qiskit and the like may be run to check Dhar's answers; they never produce them, and they are never linked into or distributed with a release. Kun keeps them in a separate class of oracle dependency, lists them separately in the bill of materials, and refuses a release in which one has reached the build (Stage-delta §3.8).

This is a security rule as much as a design rule: a dependency is code that Dhar's guarantees do not cover, and an adopter's audit would have to extend into it. The operating system's own library, used as the system interface on Windows and macOS (Amendment A-2 §3.1), is the boundary between a program and the machine it runs on. It is not a dependency of the language, and it is named as that target's interface so that a reviewer knows what a program on that target trusts.

6. Sensitive Data Handling

6.1 Constant-time operations

Comparison, branching and indexing over secret data (key material, MACs, password hashes) **must** be constant in time with respect to the secret's value, closing timing side channels of the kind that have repeatedly broken otherwise mathematically sound cryptographic implementations (CWE-208). Two things enforce the rule:

1. **The language.** A value of the secret type (Spec1 §14.5) cannot decide a branch, be an index, or be the operand of an instruction whose time depends on its value; the compiler rejects such a program with a named diagnostic. The only ways to a public value are the explicit reveal and the constant-time select. This is gate S2 (Amendment A-1 §9).
2. **The compiler.** No stage may create what the source does not contain: a select turned into a branch on a secret, a table lookup indexed by a secret, a clearing deleted (Spec1 §14.5; Stage-gamma §3.1). Each optimization pass has its own test for it.

std.crypto (Stage-delta §2) **must** be written with the secret type throughout, so that the compiler's checks apply to it. Gate S3 adds the timing test: each secret-handling routine is run on two classes of input at least a million times each, and the two timing distributions are compared with a published leakage-detection method (for example the one of dudect, by Reparaz, Balasch and Verbauwhede, 2017); a significant difference fails the build (Stage-epsilon §3.2).

Constant time is a property of the instructions Dhar emits, on an instruction set whose timing is documented (Amendment A-1 §10). It is not a promise about every microarchitectural side channel, and a statement of it names the target, the method and the run (§11). The instructions that a target's description lists as constant-time (Amendment A-1, R4) are the only ones on which a secret may be operated.

6.2 Zero-on-drop

A secret binding or buffer is overwritten with zeros, not merely deallocated, when its owner goes out of scope — closing the gap where freed-but-uncleared memory is later reused and its previous contents recovered (CWE-226). The rule is Spec1 §14.5: the compiler **must not** remove, shorten or reorder the clearing even if the value is never read again, and a copy the compiler makes itself (a spill to the stack, a temporary) is cleared on the same terms. A secret held in an arena (Spec1 §14.6) is cleared when the arena is released.

The test is the one of Stage-gamma §3.1: a program with a secret, each optimization pass applied alone, and the emitted code inspected for the clearing. Two limits are stated plainly. Clearing does not protect a secret that is still live when the operating system swaps its memory out or writes a crash dump; keeping secrets out of swap is a setting of the system, not a guarantee of the language. And a secret handed to `foreign`, `asm`, `peek`, `poke` or `sys` has left Dhar's guarantees (§3.1); this document claims nothing about it.

6.3 Post-quantum cryptography and algorithm agility

A public-key algorithm that a future quantum computer breaks puts at risk data that is encrypted now, because an adversary can record the ciphertext and decrypt it later (“harvest now, decrypt later”). The protection needs no quantum hardware: it is cryptography designed to resist a quantum computer. NIST's first three standards for it are ML-KEM (FIPS 203), ML-DSA (FIPS 204) and SLH-DSA (FIPS 205), finalized in August 2024, and the third track of Dhar begins with them (Amendment A-1, Q1). Three rules follow:

1. std.crypto **must** provide the three algorithms, written in Dhar, and every parameter set **must** pass the NIST known-answer tests (gate Q1, Amendment A-1 §9).
2. The secret-handling routines of these algorithms are held to §6.1 like every other routine of std.crypto.
3. The interfaces of std.crypto **must** let a program change algorithm without changing its structure (algorithm agility, Stage-epsilon §3.2), so that a deployment can move from a classical algorithm to a post-quantum one, or from one post-quantum algorithm to another, by configuration and a rebuild.

These standards are the work of others and can change; Dhar's part is to implement them correctly and to stay agile. This document makes no statement that any algorithm resists every future computer. The statement allowed is that a routine passes gate Q1 on the NIST vectors (§11).

7. Certification Readiness

Nothing in this section is a claim that Dhar holds a certification. A certification is granted to a finished, versioned product by a body that evaluates it, and it is never inherited from a design. This section states which frameworks are relevant, what each actually requires, and what evidence Dhar's stages and gates are to produce for an evaluation.

FRAMEWORK	WHAT IT ACTUALLY IS	WHERE DHAR STANDS
Common Criteria (ISO/IEC 15408)	International standard for security evaluation, seven Evaluation Assurance Levels (EAL1–EAL7); requires an accredited lab to evaluate a specific, finished product against a defined Protection Profile.	Applies only to a finished, versioned toolchain submitted to a lab. Spec1's exhaustiveness and bounds-checking guarantees (§3) are the kind of property an EAL evaluation would examine first, and gates S1 and S4 (Amendment A-1 §9) produce part of the evidence.
NIST SSDF (SP 800-218)	Outcome-based secure-development practices in four families (Prepare the Organization, Protect the Software, Produce Well-Secured Software, Respond to Vulnerabilities); conformance is by vendor attestation, not third-party certification; required to sell software to the US federal government under Executive Order 14028.	The Guild of Dhar (governance) and this document's §8 (vulnerability disclosure) are direct SSDF-family inputs once there is an organization to attest on behalf of; the signed, reproducible and listed builds of §5 are inputs to its software-protection practices.
FIPS 140-3	Validates specific cryptographic modules (not whole languages or toolchains) against approved algorithms (AES, SHA-2, RSA, ECDSA and similar) through NIST and the Canadian Centre for Cyber Security's joint CMVP program.	Relevant only to std.crypto, evaluated as its own module, not to Dhar the language. Passing known-answer vectors (gates S3 and Q1) is not a NIST validation and is never described as one.
DO-178C / IEC 61508 / ISO 26262	Sector-specific software assurance for airborne systems, industrial functional safety, and automotive functional safety respectively; each requires rigorous, documented traceability from requirement to code to test.	Aspirational for the embedded/hardware use case (Spec0 §5.2); would require Stage-epsilon's static analysis and Stage-delta's testing infrastructure to produce the traceability evidence these standards demand.

The summary: Dhar's language-level design choices (§3) are the kind of evidence these frameworks look for, and are made with that evaluation in mind; none of them can be pursued in earnest until Stage-epsilon produces, and Stage-delta tests, the artifact actually being evaluated. What an evaluator is given is the record of passed gates (§11) — the gate, the run and the commit — and never a description.

8. Vulnerability Disclosure & Incident Response

- The scope is the compiler, the standard library including std.crypto, Kun and its registry, and the hosted services (Stage-epsilon §3.1).
- A published, monitored channel for reporting suspected vulnerabilities, separate from ordinary bug reports, following coordinated disclosure norms (private report, agreed remediation window, public advisory on fix).
- Every fixed vulnerability affecting a released version gets a CVE and a public security advisory, cross-referencing the affected Kun package version(s) directly — a defence or critical-infrastructure adopter needs to know which locked dependency versions are affected without re-deriving it themselves.
- A security advisory states affected versions, the fixed version, and a workaround if one exists, in that order, before any narrative explanation.
- A defect in std.crypto that leaks a secret through time, output or retained memory is a vulnerability under this section. The fix ships with a test that fails without it, added to the suite of gate S3.
- A defect in the archived Stage-alpha that could hide a change in a distributed binary is a vulnerability under this section. A repair is published as a new archived version with its own recorded hash and is named in the advisory; the archive is never edited silently (Stage-alpha §5).
- An incident involving personal data that a project service holds follows the duties of The Hallmark of Dhar §7 as well as this section.

9. Acceptance Criteria

1. Every claim in §3's table is verified by at least one test in Stage-beta's suite (per Stage-beta §4) that deliberately attempts the eliminated bug class and confirms it is caught, not merely a test that the happy path works (gate S1).
2. The concurrency guarantee in §4 has a test that deliberately attempts to alias a grab reference across a spark boundary and confirms it is rejected at compile time.
3. A published vulnerability-disclosure channel and process (§8) exists before Stage-epsilon's first tagged release, not after.
4. The two-compiler check of §5.3 (gate S4) passes for every release of the compiler, with the archived Stage-alpha unchanged since it was frozen; this document is amended, not left outdated, if a change to either compiler alters what the check can show.
5. Every rule of Spec1 §14.5 has a program that breaks it and is rejected with the named diagnostic (gate S2), and every optimization pass has the test of Stage-gamma §3.1.
6. The timing test of §6.1 passes for every secret-handling routine of std.crypto, those of §6.3 included, and a failure fails the build (gate S3; Stage-epsilon §3.2).
7. The three post-quantum algorithms of §6.3 pass the NIST known-answer tests for every parameter set (gate Q1).
8. The guarantees of §3 hold alike on every committed target: the bounds-check programs of the conformance battery pass at gate G1 on each (§3.2).
9. A review of the project's public statements finds none that breaks the rule of §11: each names its gate or is worded as an aim, and a statement whose gate has stopped passing has been withdrawn.

10. Relationship to Other Volumes

THIS DOCUMENT DEFINES	RATIFIED IN	ENFORCED/TESTED IN
Memory & type safety guarantees	Spec1 §11, §14	Stage-beta §4 acceptance tests (gate S1)
Concurrency safety	Spec1 §15	Stage-beta §4 / Stage-gamma
Constant time and clearing of secrets	Spec1 §14.5	Stage-gamma §3.1; Stage-epsilon §3.2 (gates S2, S3)
Supply-chain guarantees	Stage-delta §3.7	Stage-epsilon reproducible builds & signing
The two-compiler check	Amendment A-1, S4	Stage-beta §4.1; Stage-epsilon §3.2
Post-quantum cryptography	Amendment A-1, Q1	Stage-delta §2.1 (std.crypto); gate Q1
Hosted-service safeguards	Stage-epsilon §3.1	Stage-epsilon §4
Certification evidence	(this document, §7)	Stage-epsilon static analysis & Stage-delta testing
Claims and gates	(this document, §11); Master Index §3.6	The Progress Reports; every public statement

11. Claims and Gates

11.1 The rule

A statement about Dhar's security, speed or correctness, or about the processors and operating systems it runs on, is allowed only if it names the acceptance gate that backs it (Amendment A-1 §9; Amendment A-2 §6) or is worded as an aim. This is the rule of Master Index §3.6, and this section is its home. The words “secure”, “safe” and “fast” are never used alone.

An aim states a purpose and no result: “Dhar aims to…”, “the design requires…”. A claim states a result, and a result has a test, a reference the test is checked against, and a commit and a run where it passed. The Canon states what must hold and contains no result. Results are in the Progress Reports (Master Index §2), and a public statement that quotes one cites its entry. A statement that cannot name its entry is removed or reworded as an aim.

11.2 The allowed forms

A STATEMENT ABOUT	ALLOWED FORM	GATE
Memory safety	“Passes gate S1 on this suite, at this commit.”	S1
Constant time	“The routine R passes the timing test of gate S3 on target T, by method M, at this commit.”	S2, S3
Correctness of an algorithm	“Passes the known-answer vectors of” the named standard, for example RFC 8439.	S3, Q1

A STATEMENT ABOUT	ALLOWED FORM	GATE
A trustworthy build	“The release binary passes the two-compiler check of gate S4.”	S4
Floating point	“Bit-identical to the platform's IEEE 754 results on the edge values of gate A1.”	A1
Inference	“Predictions agree with the reference framework to tolerance T on this test set.”	A3
Speed	“On matrix multiplication and the reference network, the ratio to optimised C (`-O3`) on this machine is X, by this method.”	A4
Quantum programs	“Writes OpenQASM 3 that a reference tool accepts and simulates to the expected result on this set of circuits”; “amplitudes agree with a reference simulator up to N qubits.”	Q2, Q3
Post-quantum cryptography	“Implements ML-KEM as specified in FIPS 203 and passes the NIST known-answer tests.”	Q1
A target	“Passes G1 on target T, in this run”; when only an emulator has run it, “passes G0 on target T.”	G0, G1

11.3 What is never said

- “Secure”, “safe” or “fast” alone, and words that promise more than a gate can: “unhackable”, “bug-free”, “military-grade”, “quantum-proof”.
- “Formally verified”, or any statement that Dhar is. The non-goals of Amendment A-1 §10 hold here.
- That Dhar supports a target whose gate G1 has not passed. A run in an emulator never gives “supported”.
- That Dhar is certified, or compliant with a framework, without the issuer's certificate or an attestation under that framework's own rules (§7).
- That the in-browser playground keeps no source code, unless the test of Stage-epsilon §4, criterion 8 passes for the version in use.
- Anything about a third party's software, or about a program someone else wrote in Dhar, that the project has not itself tested.

11.4 When a gate stops passing

A claim is as old as its last passing run. When a change makes a gate fail, the claim it backs is withdrawn from public statements until the gate passes again. If a user could have relied on the withdrawn claim, §8 applies: the change is announced with an advisory, not merely corrected.

11.5 Changing the rule

This section changes only by amendment (Master Index §10), through the Guild of Dhar's process for a rule of this kind. A gate is added, changed or withdrawn only by an amendment to Amendment A-1 or A-2, never by a release note, a talk or a web page.

Revision log

Tempering Dhar — Security, Assurance & Defence-Grade Engineering · v1.1 · October 26, 2026 · Ratified. Grounded in NSA/CISA memory-safety guidance (2022–2025), NIST SP 800-218 (SSDF v1.1, Feb 2022), FIPS 140-3 (effective 2019), NIST FIPS 203, 204 and 205 (August 2024), and Common Criteria (ISO/IEC 15408).

VERSION	DATE	CHANGE
v1.0	September 8, 2026	first ratified edition; companion amendment to Spec1 §15 (view/grab exclusivity across spark/draw) made concurrently.
v1.1	October 26, 2026	Amendments A-1 and A-2: the rule for claims made a section of its own (§11), the threat model widened (§2, §3.2, §3.3), the two-compiler check specified as gate S4 (§5.3, §5.4), constant time, clearing and post-quantum cryptography specified (§6), acceptance criteria extended (§9), and Spec1 cross-references corrected.